

```

1  """Shared generative EDA: inverted-U bins + the Assign→Score→Select pipeline runner.
2
3  =====
4  FOR LATER CHARLES – read this like a notebook CELL map
5  =====
6  Daily path: 540_READ_ME_SIM.md + hero_model_reset_bundle / 540_rho_ablation_bundle.
7  Plain English: 3-Master_Plan/re_entry/04_Pass_A_and_Pass_B_in_Plain_English.md
8
9  This file sits *above* the engine (tier1_pool_assignment.py):
10 • knobs come from tier1_sim_config.py (or CELL 10 state JSON)
11 • ASSIGN / SCORE / SELECT math lives in tier1_pool_assignment
12 • HERE we bundle selection knobs, build the binned inverted-U table,
13   draw Plot B, and run one full preview pipeline
14
15 -----
16 READ-FIRST GLOSSARY – defined HERE before any code uses these words.
17 (Same letters as tier1_pool_assignment / tier1_sim_config – repeated so you
18 do not have to jump files mid-read.)
19 -----
20
21 WHAT THIS FILE DOES
22 Takes a synthetic roster (or builds one), scores/selects players, then
23 summarizes selection *rate* by bins of pool quality – the sim “inverted-U”
24 table and figure Pass A/B bundles look at.
25
26 THREE STEPS (engine does 1–3; this file orchestrates + bins/plots)
27 (1) ASSIGN – seat players (simulate_generative_rosters).
28 (2) SCORE –  $S_i$  from ability  $\pm$  pool  $L$  (assign_selection → selection_weights).
29 (3) SELECT – winners  $Y_{\text{selected}}$  (usually top  $K$ , choice “C”).
30 Then: BIN + PLOT – mean  $Y_{\text{selected}}$  vs bin of  $L_Q$  (or team mean).
31
32 CORE LETTERS
33  $A_i$ ,  $T_j$ , pool,  $S_i$ ,  $K$ ,  $Y_{\text{selected}}$  – same as the engine glossary.
34  $L$  /  $LOO$  /  $L_Q$  /  $L_C$  /  $\theta$  /  $\gamma$  /  $w$  /  $\lambda$  /  $q$  – same as the engine glossary.
35  $l_{\text{term\_scale}}$  / CROWDING  $L_Z$  SCALE – unit matcher for crowding  $L$ ; decode in
36 tier1_pool_assignment READ-FIRST GLOSSARY ( $l = L$ ; term = L-piece of score).
37
38 IMPORTANT AXIS RULE (easy to miss)
39 Plot B / inverted_u_bin_table X-axis is ALWAYS  $L_Q$  (poolq_loo) by default –
40 mean teammate ability – so curves stay comparable when you change which  $L$ 
41 enters the *score*. Changing loo_pool_l_mode changes SCORE, not the bin axis
42 (unless you deliberately use the team_mean Plot B’ helper).
43
44 SELECTIONCONFIG vs ASSIGNMENTPARAMS
45 AssignmentParams – league geometry + assign knobs (from tier1_sim_config).
46 SelectionConfig – bins,  $K$ , score mode,  $w$ , winner rule, which  $L$  in score.
47 Pass A mostly varies SelectionConfig (and  $L$  mode /  $w$ ).
48 Pass B mostly varies AssignmentParams.assignment_rho / method.
49
50 SECTION MAP
51 0. Outcome column helper ( $Y_{\text{selected}}$  vs legacy  $Y_{\text{promoted}}$ )
52 1. SelectionConfig – SCORE/SELECT + binning knobs
53 2. Plot B axis label strings
54 3. Formula / title helpers (what the figure claims)
55 4. Load knobs from CELL 10 state JSON (legacy playground)
56 5. inverted_u_bin_table – binned selection rates (main Plot B data)
57 6. inverted_u_bin_table_team_mean – optional 539-style X-axis
58 7. figure_inverted_u – draw Plot B
59 8. run_inverted_u_pipeline – one-shot Assign→Select→bin→figure
60 =====
61 """
62
63 from __future__ import annotations
64
65 import json
66 from dataclasses import dataclass
67 from pathlib import Path
68 from typing import TYPE_CHECKING
69
70 import matplotlib.pyplot as plt
71 import numpy as np
72 import pandas as pd
73
74 if TYPE_CHECKING:
75     from tier1_pool_assignment import AssignmentParams
76

```

```

77
78 # =====
79 # 0. OUTCOME COLUMN HELPER
80 # =====
81 # Early drafts called the 0/1 column Y_promoted; current name is Y_selected.
82 # Same meaning: 1 = got the slot after SELECT.
83
84
85 def _outcome_col(players: pd.DataFrame) -> str:
86     """Return the 0/1 outcome column name on a scored roster table.
87
88     Prefers Y_selected; falls back to legacy Y_promoted.
89     """
90     if "Y_selected" in players.columns:
91         return "Y_selected"
92     if "Y_promoted" in players.columns:
93         return "Y_promoted"
94     raise KeyError("players need Y_selected or Y_promoted")
95
96
97 # =====
98 # 1. SelectionConfig - SCORE / SELECT / BINNING KNOBS
99 # =====
100 # AssignmentParams (engine) = how the fake league is built and who sits where.
101 # SelectionConfig (this class) = how we score, how many we pick, how we bin Plot B.
102
103
104 @dataclass(frozen=True)
105 class SelectionConfig:
106     """Knobs for SCORE + SELECT + inverted-U binning (mirror tier1_sim_config).
107
108     Field-by-field (530 CELL 2 style)
109     n_bins          - how many bins on the Plot B X-axis.
110     bin_mode        - "quantile" or "equal_width" cuts on that axis.
111     n_selected (K)  - how many players get Y_selected=1.
112     score_mode      - "ability" or "loo_gap_plus_ability" (Pass A toggle).
113     loo_gap_weight  - w in S = A - w*L (same role as Alex λ).
114     winner_selection - "A"/"B"/"C"; Pass A/B default "C" = top K by score.
115     loo_pool_l_mode - which L enters the SCORE (quality / crowding / ...).
116                     Does NOT change the default Plot B bin axis (still L_Q).
117     """
118
119     n_bins: int
120     # n_bins: number of X-axis bins for the inverted-U / Plot B summary.
121     bin_mode: str
122     # bin_mode: "quantile" (~equal n per bin) or "equal_width" (equal L spacing).
123     n_selected: int
124     # n_selected (K): how many winners under the winner rule.
125     score_mode: str
126     # score_mode: "ability" (S=A) or "loo_gap_plus_ability" (S=A-w*L).
127     loo_gap_weight: float
128     # loo_gap_weight (w): weight on the L-term; = Alex λ in the nesting.
129     winner_selection: str
130     # winner_selection: "C" = top K (default); "A"/"B" = stochastic (legacy).
131     loo_pool_l_mode: str = "quality"
132     # loo_pool_l_mode: which pool L enters SCORE - not the default Plot B X-axis.
133
134     @classmethod
135     def from_module(cls, mod) -> SelectionConfig:
136         """Build from an imported tier1_sim_config (or compatible) module.
137
138         Reads N_SELECTED / SELECTION_SCORE_MODE with legacy N_PROMOTED /
139         PROMOTION_SCORE_MODE fallbacks so old configs still load.
140         """
141         n_sel = getattr(mod, "N_SELECTED", None)
142         if n_sel is None:
143             n_sel = getattr(mod, "N_PROMOTED", 40)
144         score = getattr(mod, "SELECTION_SCORE_MODE", None)
145         if score is None:
146             score = getattr(mod, "PROMOTION_SCORE_MODE", "loo_gap_plus_ability")
147         return cls(
148             n_bins=int(getattr(mod, "GENERATIVE_N_BINS", 12)),
149             bin_mode=str(getattr(mod, "GENERATIVE_POOLQ_BINNING", "quantile")),
150             n_selected=int(n_sel),
151             score_mode=str(score),
152             loo_gap_weight=float(getattr(mod, "LOO_GAP_WEIGHT", 0.5)),
153             winner_selection=str(getattr(mod, "WINNER_SELECTION", "C")),
154             loo_pool_l_mode=str(getattr(mod, "LOO_POOL_L_MODE", "quality")),
155         )
156
157     @classmethod
158     def from_state(cls, state: dict, base: SelectionConfig) -> SelectionConfig:

```

```

159     """Overlay CELL 10 playground JSON knobs onto a base SelectionConfig.
160
161     state keys use short names (n_selected, score_mode, ...) from the widget UI.
162     Missing keys keep the base value.
163     """
164     n_sel = state.get("n_selected", state.get("n_promoted", base.n_selected))
165     return cls(
166         n_bins=int(state.get("n_bins", base.n_bins)),
167         bin_mode=str(state.get("bin_mode", base.bin_mode)),
168         n_selected=int(n_sel),
169         score_mode=str(state.get("score_mode", base.score_mode)),
170         loo_gap_weight=float(state.get("loo_gap_weight", base.loo_gap_weight)),
171         winner_selection=str(state.get("winner_selection", base.winner_selection)),
172         loo_pool_l_mode=str(
173             state.get("loo_pool_l_mode", base.loo_pool_l_mode)
174         ),
175     )
176
177
178 # Deprecated alias - early drafts said "PromotionConfig"; same class.
179 PromotionConfig = SelectionConfig
180
181
182 # =====
183 # 2. PLOT B AXIS LABEL STRINGS
184 # =====
185 # Default X = L_Q bins (530-comparable). Optional team_mean axis = 539-style.
186
187 # PLOT_B_XAXIS_LABEL: default X - bin mean of L_Q (LOO mean teammate ability).
188 PLOT_B_XAXIS_LABEL = (
189     "Bin mean L_Q (LOO mean teammate ability)\n"
190     "[x-axis fixed - Pool L dropdown affects selection score only]"
191 )
192 # PLOT_B_XAXIS_TEAM_MEAN_LABEL: optional 539-style X - realized team mean ability.
193 PLOT_B_XAXIS_TEAM_MEAN_LABEL = (
194     "Bin mean team ability (pool mean; 539-style)\n"
195     "[Pool L dropdown affects selection score only]"
196 )
197 # PLOT_B_YAXIS_LABEL: Y - mean of Y_selected inside each bin (selection rate).
198 PLOT_B_YAXIS_LABEL = "Mean selection rate (Y_selected) per bin"
199
200
201 # =====
202 # 3. FORMULA / TITLE HELPERS (what the figure claims in plain text)
203 # =====
204
205
206 def selection_rank_formula(
207     score_mode: str,
208     *,
209     loo_gap_weight: float,
210     pool_l_mode: str,
211     tpa=None,
212 ) -> str:
213     """Plain-text formula for who wins top-K (not the Plot B bin axis).
214
215     Args
216         score_mode      - "ability" or "loo_gap_plus_ability".
217         loo_gap_weight  - w / λ in S = A - w·L.
218         pool_l_mode     - which L enters the score (quality / crowding_smooth...).
219         tpa             - optional tier1_pool_assignment module (imported if None).
220
221     Returns a one-line string suitable for a figure title subtitle.
222     """
223     if tpa is None:
224         import tier1_pool_assignment as tpa # noqa: PLC0415
225
226     mode = str(score_mode).strip().lower()
227     if mode == "ability":
228         return "Selection rank = A_i only (top-K; w unused)"
229     w = float(loo_gap_weight)
230     l_short = tpa.pool_l_short_label(pool_l_mode)
231     if tpa.is_crowding_l_mode(pool_l_mode):
232         return (
233             f"Selection rank = A_i - w·{l_short} "
234             f"(w={w:.2f}; Pool L in score={pool_l_mode!r})"
235         )
236     return f"Selection rank = w·(A_i-L_Q)+(1-w)·A_i (w={w:.2f})"
237
238
239 def plot_b_figure_title(
240     sel: SelectionConfig,

```

```

241 *,
242 header: str = "538 CELL 10 - Plot B",
243 team_mean_axis: bool = False,
244 tpa=None,
245 ) -> str:
246     """Two-line matplotlib title: y vs Plot B bins + selection-rank recipe.
247
248     Args
249         sel                - SelectionConfig (K, bins, score mode, L mode, w).
250         header             - first-line prefix (Pass A/B bundles customize this).
251         team_mean_axis     - True if X is team_mean bins (539-style), else L_Q bins.
252         tpa                - optional tier1_pool_assignment module.
253     """
254     rank = selection_rank_formula(
255         sel.score_mode,
256         loo_gap_weight=sel.loo_gap_weight,
257         pool_l_mode=sel.loo_pool_l_mode,
258         tpa=tpa,
259     )
260     x_label = "team_mean bins (539-style)" if team_mean_axis else "L_Q bins"
261     return (
262         f"{header}: mean Y_selected vs {x_label}\n"
263         f"{rank} | K={sel.n_selected} | {sel.n_bins} bins ({sel.bin_mode})"
264     )
265
266
267 # =====
268 # 4. LOAD KNOBS FROM CELL 10 STATE JSON (legacy playground)
269 # =====
270 # Daily 540 re-entry prefers scripts + tier1_sim_config defaults.
271 # These helpers still support the old CELL 10 widget state file.
272
273
274 def assignment_params_from_state(
275     sports: Path,
276     state: dict | None,
277     *,
278     tpa,
279 ) -> AssignmentParams:
280     """Build AssignmentParams from tier1_sim_config, then overlay playground state.
281
282     Args
283         sports - path to the sports/ directory (holds tier1_sim_config.py).
284         state  - dict from load_playground_state (or None / {} -> config defaults).
285         tpa    - tier1_pool_assignment module (caller imports it).
286
287     State key map (widget short name -> AssignmentParams field)
288         n_teams, roster_size, target_dist, t_low, t_high, kernel, tau, rho, sigma,
289         use_preferential_attachment, pref_alpha, ability_draw,
290         viability_theta, viability_sharpness
291     """
292     base = tpa.AssignmentParams.from_tier1_sim_config(sports / "tier1_sim_config.py")
293     if not state:
294         return base
295     return tpa.AssignmentParams(
296         n_teams=int(state.get("n_teams", base.n_teams)),
297         roster_size=int(state.get("roster_size", base.roster_size)),
298         target_mean_dist=state.get("target_dist", base.target_mean_dist),
299         target_mean_low=float(state.get("t_low", base.target_mean_low)),
300         target_mean_high=float(state.get("t_high", base.target_mean_high)),
301         target_mean_mu=base.target_mean_mu,
302         target_mean_sigma=base.target_mean_sigma,
303         assignment_kernel=state.get("kernel", base.assignment_kernel),
304         assignment_temperature=float(state.get("tau", base.assignment_temperature)),
305         assignment_rho=float(state.get("rho", base.assignment_rho)),
306         assignment_sigma=float(state.get("sigma", base.assignment_sigma)),
307         use_preferential_attachment=bool(
308             state.get("use_preferential_attachment", base.use_preferential_attachment)
309         ),
310         preferential_alpha=float(state.get("pref_alpha", base.preferential_alpha)),
311         preferential_k=base.preferential_k,
312         ability_draw=state.get("ability_draw", base.ability_draw),
313         ability_mean=base.ability_mean,
314         ability_sd=base.ability_sd,
315         ability_clip_low=base.ability_clip_low,
316         ability_clip_high=base.ability_clip_high,
317         ability_student_t_df=base.ability_student_t_df,
318         ability_student_t_scale=base.ability_student_t_scale,
319         sorting_noise_sd=base.sorting_noise_sd,
320         viability_theta=float(
321             state.get("viability_theta", base.viability_theta)
322         ),

```

```

323     viability_sharpness=float(
324         state.get("viability_sharpness", base.viability_sharpness)
325     ),
326 )
327
328
329 def load_playground_state(sports: Path) -> dict:
330     """Load sports/tier1_cell10_playground_state.json if present; else {}.
331
332     Missing or corrupt file → empty dict (callers fall back to config defaults).
333     Not on the daily 540 checklist path – legacy CELL 10 UI state only.
334     """
335     path = sports / "tier1_cell10_playground_state.json"
336     if not path.is_file():
337         return {}
338     try:
339         return json.loads(path.read_text(encoding="utf-8"))
340     except (OSError, json.JSONDecodeError):
341         return {}
342
343
344 # =====
345 # 5. inverted_u_bin_table – MAIN PLOT B DATA (X = L_Q always)
346 # =====
347
348
349 def inverted_u_bin_table(
350     players: pd.DataFrame,
351     sel: SelectionConfig,
352     *,
353     assign_poolq_bin_labels,
354     tpa=None,
355 ) -> pd.DataFrame:
356     """Binned inverted-U table: X-axis always L_Q (`poolq_loo`), not crowding.
357
358     What each output column *is*
359     bin                – bin label from assign_poolq_bin_labels
360     n                  – how many players in the bin
361     selection_rate      – mean Y_selected (or Y_promoted) in the bin
362     mean_loo_q          – mean L_Q inside the bin (Plot B X)
363
364     Args
365     players – roster table that already has LOO columns + Y_selected
366               (after assign_selection), or at least poolq_loo + outcome.
367     sel      – SelectionConfig (n_bins, bin_mode; L mode does NOT move X).
368     assign_poolq_bin_labels – callable(series, n_bins, bin_mode) → bin labels
369               (usually from sports_pipeline / notebook helper).
370     tpa      – optional tier1_pool_assignment module.
371
372     Reminder: sel.loo_pool_l_mode affects the selection *score* only (upstream).
373     Bins stay on LOO mean teammate ability so Plot B stays comparable across
374     quality vs crowding score modes.
375     """
376     if tpa is None:
377         import tier1_pool_assignment as tpa # noqa: PLC0415
378
379     ycol = _outcome_col(players)
380     # bin_lcol: always L_Q column name – fixed X-axis by design.
381     bin_lcol = tpa.POOL_L_QUALITY_COL
382     use = players.dropna(subset=[bin_lcol, ycol]).copy()
383     use["bin"] = assign_poolq_bin_labels(use[bin_lcol], sel.n_bins, sel.bin_mode)
384     return (
385         use.dropna(subset=["bin"])
386         .groupby("bin", observed=True)
387         .agg(
388             n=(ycol, "size"),
389             selection_rate=(ycol, "mean"),
390             mean_loo_q=(bin_lcol, "mean"),
391         )
392         .reset_index()
393         .sort_values("mean_loo_q")
394     )
395
396
397 # =====
398 # 6. inverted_u_bin_table_team_mean – OPTIONAL 539-STYLE X-AXIS
399 # =====
400
401
402 def inverted_u_bin_table_team_mean(
403     players: pd.DataFrame,
404     sel: SelectionConfig,

```

```

405     *,
406     assign_poolq_bin_labels,
407 ) -> pd.DataFrame:
408     """539-style Plot B': binned selection rate vs realized pool (team) mean ability.
409
410     Differs from inverted_u_bin_table: X is each team's mean A (including self),
411     not LOO L_Q. Use only when you deliberately want the 539 notebook axis.
412     """
413     ycol = _outcome_col(players)
414     use = players.copy()
415     # pool_mean: average ability on this player's team (realized roster mean).
416     use["pool_mean"] = use.groupby("pool_id", observed=True)["ability"].transform("mean")
417     use = use.dropna(subset=["pool_mean", ycol])
418     use["bin"] = assign_poolq_bin_labels(use["pool_mean"], sel.n_bins, sel.bin_mode)
419     return (
420         use.dropna(subset=["bin"])
421         .groupby("bin", observed=True)
422         .agg(
423             n=(ycol, "size"),
424             selection_rate=(ycol, "mean"),
425             mean_team_mean=("pool_mean", "mean"),
426         )
427         .reset_index()
428         .sort_values("mean_team_mean")
429     )
430
431
432 # =====
433 # 7. figure_inverted_u - DRAW PLOT B
434 # =====
435
436
437 def figure_inverted_u(
438     summ: pd.DataFrame,
439     *,
440     title: str,
441     n_bins: int,
442     n_teams: int,
443     show_bin_n: bool = True,
444     loo_pool_l_mode: str = "quality",
445     x_col: str | None = None,
446     xlabel: str | None = None,
447     tpa=None,
448 ) -> plt.Figure:
449     """Draw the inverted-U / Plot B line from a bin summary table.
450
451     Args
452         summ            - output of inverted_u_bin_table(_team_mean).
453         title           - full matplotlib title (often from plot_b_figure_title).
454         n_bins / n_teams- kept for call-site clarity / future annotations
455                         (not all are drawn on the axes today).
456         show_bin_n      - if True, print n=... above each point.
457         loo_pool_l_mode - unused for X; kept so callers can pass sel fields.
458         x_col           - force which summ column is X; else auto-detect
459                         mean_team_mean / mean_loo_q / legacy mean_loo_l.
460         xlabel          - X axis label string; default PLOT_B_XAXIS_LABEL.
461         tpa             - optional tier1_pool_assignment (imported if needed later).
462     """
463     if tpa is None:
464         import tier1_pool_assignment as tpa # noqa: PLC0415
465
466     fig, ax = plt.subplots(figsize=(8.5, 4.2))
467     # Pick X column: explicit override, else team_mean, else L_Q, else legacy name.
468     if x_col is not None:
469         xcol = x_col
470     elif "mean_team_mean" in summ.columns:
471         xcol = "mean_team_mean"
472     elif "mean_loo_q" in summ.columns:
473         xcol = "mean_loo_q"
474     elif "mean_loo_l" in summ.columns:
475         xcol = "mean_loo_l"
476     else:
477         raise KeyError("summ needs mean_loo_q, mean_team_mean, or legacy mean_loo_l")
478     x = summ[xcol].to_numpy(dtype=float)
479     # rate_col: selection_rate (current) or promo_rate (legacy column name).
480     rate_col = (
481         "selection_rate"
482         if "selection_rate" in summ.columns
483         else "promo_rate"
484     )
485     y = summ[rate_col].to_numpy(dtype=float)
486     ax.plot(x, y, "o-", color="C0", lw=2.0, ms=7)

```



```

487 ax.fill_between(x, 0, y, alpha=0.12, color="C0")
488 ax.set_xlabel(xlabel or PLOT_B_XAXIS_LABEL)
489 ax.set_ylabel(PLOT_B_YAXIS_LABEL)
490 ax.set_title(title, fontsize=10)
491 ymax = float(y.max()) if len(y) else 0.0
492 ymin = float(y.min()) if len(y) else 0.0
493 if ymax <= 0:
494     ax.set_ylim(0, 0.01)
495 else:
496     span = ymax - max(ymin, 0.0)
497     pad = max(span * 0.12, ymax * 0.08, 1e-4)
498     ax.set_ylim(0, min(1.0, ymax + pad))
499
500 if show_bin_n and "n" in summ.columns and len(summ):
501     y_top = ax.get_ylim()[1]
502     for xi, yi, ni in zip(x, y, summ["n"].to_numpy(dtype=int), strict=True):
503         label_y = min(float(yi) + y_top * 0.04, y_top * 0.98)
504         ax.text(
505             xi,
506             label_y,
507             f"n={int(ni):,}",
508             ha="center",
509             va="bottom",
510             fontsize=7,
511             color="0.35",
512             clip_on=True,
513         )
514
515 fig.tight_layout()
516 return fig
517
518
519 # =====
520 # 8. run_inverted_u_pipeline - ONE-SHOT ASSIGN → SELECT → BIN → FIGURE
521 # =====
522
523
524 def run_inverted_u_pipeline(
525     params: AssignmentParams,
526     sel: SelectionConfig,
527     rng,
528     *,
529     tpa,
530     assign_poolq_bin_labels,
531     method: str = "soft",
532     ability: np.ndarray | None = None,
533     team_targets: np.ndarray | None = None,
534 ) -> tuple[pd.DataFrame, pd.DataFrame, plt.Figure]:
535     """Full preview: ASSIGN → SCORE/SELECT → L_Q bin table → Plot B figure.
536
537     This is the thin "conductor" Pass A/B-style previews call. Heavy math is in
538     tier1_pool_assignment; this function only sequences the steps.
539
540     Args
541         params - AssignmentParams (league + assign knobs; Pass B varies q here).
542         sel - SelectionConfig (K, score mode, w, L mode; Pass A varies these).
543         rng - NumPy Generator (caller owns the seed).
544         tpa - tier1_pool_assignment module.
545         assign_poolq_bin_labels - binning helper (same as inverted_u_bin_table).
546         method - "soft" or "sort_chop" (Pass B extreme arm).
547         ability / team_targets - optional pre-drawn A_i / T_j so Pass B reuses
548             the same talent deck across q arms.
549
550     Returns
551         players - roster with L00 columns, selection_weight, Y_selected
552         summ - inverted_u_bin_table (n, selection_rate, mean_loo_q per bin)
553         fig - matplotlib Figure for Plot B
554     """
555     # Step 1 - ASSIGN: build fake rosters (no Y_selected yet).
556     players, _, _ = tpa.simulate_generative_rosters(
557         params,
558         rng=rng,
559         method=method,
560         ability=ability,
561         team_targets=team_targets,
562     )
563     # Steps 2-3 - SCORE then SELECT (adds L columns, S_i, Y_selected).
564     players = tpa.assign_selection(
565         players,
566         rng,
567         n_selected=sel.n_selected,
568         score_mode=sel.score_mode,

```

```

569         loo_gap_weight=sel.loo_gap_weight,
570         winner_selection=sel.winner_selection,
571         pool_l_mode=sel.loo_pool_l_mode,
572         viability_theta=params.viability_theta,
573         viability_sharpness=params.viability_sharpness,
574     )
575     # Bin on L_Q and draw Plot B.
576     summ = inverted_u_bin_table(
577         players, sel, assign_poolq_bin_labels=assign_poolq_bin_labels, tpa=tpa
578     )
579     fig = figure_inverted_u(
580         summ,
581         title=plot_b_figure_title(
582             sel,
583             header=f"Inverted-U preview (J={params.n_teams})",
584             tpa=tpa,
585         ),
586         n_bins=sel.n_bins,
587         n_teams=params.n_teams,
588         loo_pool_l_mode=sel.loo_pool_l_mode,
589         tpa=tpa,
590     )
591     return players, summ, fig

```